Séance 04 - Parcourir des tableaux avec `foreach`

Table of Contents

1. Situation professionnelle . . . 2
2. Objectifs . . . 2
3. Prérequis . . . 2
4. Réactivation . . . 2
5. Activité 1 - Préparer l'environnement . . . 3
6. Activité 2 - Comprendre les trois boucles . . . 3
   6.1. Aperçu de `for` . . . 3
   6.2. Aperçu de `while` . . . 4
   6.3. Choisir . . . 4
7. Activité 3 - Découvrir les tableaux . . . 4
   7.1. Tableau indexé . . . 4
   7.2. Tableau associatif . . . 5
8. Activité 4 - Parcourir un tableau indexé avec `foreach` . . . 6
   8.1. Comprendre la syntaxe . . . 6
   8.2. Prévoir une trace . . . 6
   8.3. Transformer l'affichage . . . 7
   8.4. Premier commit de la séance . . . 7
9. Activité 5 - Parcourir les clés et les valeurs . . . 7
   9.1. Construire une phrase . . . 8
10. Activité 6 - Combiner `foreach` et conditions . . . 8
11. Activité 7 - Compter, cumuler et calculer . . . 9
12. Activité 8 - Diagnostiquer un parcours . . . 10
13. Exercice autonome - Produire un rapport de stockage . . . 11
14. Travail sur papier . . . 12
15. Exercices bonus . . . 12
    15.1. Exercice bonus 1 `[*]` - Le serveur le plus chargé . . . 13
    15.2. Exercice bonus 2 `[**]` - Amplitude des charges . . . 13
    15.3. Exercice bonus 3 `[*]` - Défi : un rapport qui ne plante pas . . . 13
16. Synthèse . . . 13
17. Validation Git . . . 14
18. Arrêter le serveur PHP . . . 14

1. Situation professionnelle

Vous poursuivez le développement d'un outil de supervision d'un parc informatique.

Un programme doit souvent appliquer le même traitement à plusieurs serveurs, services ou mesures. Ces données seront regroupées dans des tableaux, puis parcourues principalement avec la boucle `foreach`.

La séance présente brièvement `for` et `while`, mais se concentre sur le parcours des tableaux avec `foreach`.

2. Objectifs

À la fin de la séance, vous devrez être capable de :

- expliquer le rôle général d'une boucle ;
- reconnaître les usages de `for`, `while` et `foreach` ;
- déclarer un tableau indexé et un tableau associatif simples ;
- inspecter un tableau avec `print_r()` ;
- parcourir les valeurs d'un tableau indexé ;
- parcourir les clés et les valeurs d'un tableau associatif ;
- filtrer, compter et cumuler des valeurs pendant un parcours ;
- prévoir la trace d'un `foreach` ;
- enregistrer le travail avec Git.

3. Prérequis

- Les séances 01 à 03 sont terminées.
- Vous avez le dossier du TP (avec `compose.yml`, `index.php` et `diagnostic.php`).
- Docker Desktop est démarré (icône baleine).



Rappel du démarrage : ouvrez le dossier du TP, tapez `docker compose up -d`, puis ouvrez `http://localhost:8181`. Vos fichiers `.php` vont **dans le dossier du TP**. Guide pas à pas : `cours/annexe-docker-php.adoc`.

4. Réactivation

Sans ordinateur, répondez aux questions.

1. Quelle valeur produit l'expression `75 >= 70` ?
2. Quelle structure permet de choisir entre plusieurs traitements ?

3. Quel opérateur ajoute `1` à une variable ?
4. Quelle commande Git affiche les fichiers modifiés ?
5. Quelle commande Docker affiche l'état du service PHP ?

5. Activité 1 - Préparer l'environnement

Depuis la racine du dépôt étudiant :

```
git status
git pull
```

Vérifiez que les séances précédentes sont présentes et qu'aucun travail local ne risque d'être écrasé.

Dans le dossier du TP, démarrez le serveur (s'il ne tourne pas déjà) :

```
docker compose up -d
```

Vérifiez `http://localhost:8181/`. Vos fichiers `.php` se créent dans le dossier du TP et s'ouvrent à une adresse de la forme :

```
http://localhost:8181/nom-du-fichier.php
```

6. Activité 2 - Comprendre les trois boucles

Une boucle répète un bloc d'instructions tant qu'une règle l'autorise.

Boucle	Usage privilégié	Question à se poser
`for`	Répéter un nombre connu de fois.	« Combien de répétitions sont prévues ? »
`while`	Répéter tant qu'un état reste vrai.	« Qu'est-ce qui fera devenir la condition fausse ? »
`foreach`	Parcourir les éléments d'un tableau.	« Quelle variable recevra l'élément courant ? »

6.1. Aperçu de `for`

```
for ($numero = 1; $numero <= 3; $numero++) {
    echo "Contrôle " . $numero . "<br>";
}
```

Le code affiche trois contrôles. Les trois parties de l'en-tête initialisent la variable, testent la

condition et la font évoluer.

6.2. Aperçu de `while`

```
$tentative = 1;

while ($tentative <= 3) {
    echo "Tentative " . $tentative . "<br>";
    $tentative++;
}
```



Dans un `while`, une instruction doit rapprocher la variable testée de la condition d'arrêt. Sans `$tentative++`, cette boucle serait infinie.

6.3. Choisir

Associez chaque besoin à la boucle la plus naturelle.

Besoin	Boucle
Afficher les numéros de 1 à 12	…
Retenter une opération tant qu'elle échoue	…
Afficher tous les serveurs d'un tableau	…
Traiter toutes les notes associées à des étudiants	…



Plusieurs solutions sont parfois possibles. Dans ce module, `foreach` sera utilisé par défaut pour parcourir un tableau.

7. Activité 3 - Découvrir les tableaux

Un tableau est une variable qui contient plusieurs valeurs.

7.1. Tableau indexé

Créez `tableaux.php` :

```
<?php

$serveurs = ["srv-web-01", "srv-bdd-01", "srv-cache-01"];

echo "<pre>";
print_r($serveurs);
echo "</pre>";
```

PHP associe automatiquement un indice à chaque valeur. Le premier indice est `0`.

Expression	Valeur
`$serveurs[0]`	`"srv-web-01"`
`$serveurs[1]`	`"srv-bdd-01"`
`$serveurs[2]`	`"srv-cache-01"`

Ajoutez un élément :

```
$serveurs[] = "srv-fichiers-01";
```

Affichez de nouveau le tableau, puis affichez son nombre d'éléments :

```
echo "Nombre de serveurs : " . count($serveurs);
```

7.2. Tableau associatif

Un tableau associatif relie une clé à une valeur.

Ajoutez dans `tableaux.php` :

```
$chargesCpu = [
    "srv-web-01" => 45,
    "srv-bdd-01" => 82,
    "srv-cache-01" => 28
];

echo "<pre>";
print_r($chargesCpu);
echo "</pre>";

echo "Charge de srv-bdd-01 : " . $chargesCpu["srv-bdd-01"] . " %";
```

Dans ce tableau :

- les noms des serveurs sont les clés ;
- les charges CPU sont les valeurs ;
- la clé permet d'accéder directement à une valeur.

QUESTION: Quelle différence observez-vous entre l'indice `0` du premier tableau et la clé `"srv-web-01"` du second ?

8. Activité 4 - Parcourir un tableau indexé avec `foreach`

8.1. Comprendre la syntaxe

Créez `liste-serveurs.php` :

```
<?php

$serveurs = ["srv-web-01", "srv-bdd-01", "srv-cache-01"];

foreach ($serveurs as $serveur) {
    echo "Vérification de " . $serveur . "<br>";
}
```

À chaque répétition :

1. PHP prend la valeur suivante dans `$serveurs` ;
2. cette valeur est placée temporairement dans `$serveur` ;
3. le bloc entre accolades est exécuté ;
4. le parcours s'arrête automatiquement après le dernier élément.

La variable `$serveur` est choisie par le développeur. Son nom doit décrire un élément du tableau.

Nom du tableau	Nom conseillé pour l'élément
`$serveurs`	`$serveur`
`$services`	`$service`
`$utilisateurs`	`$utilisateur`
`$notes`	`$note`

8.2. Prévoir une trace

Complétez avant l'exécution.

Répétition	Valeur de `$serveur`	Texte affiché
1	…	…
2	…	…
3	…	…

QUESTION: Pourquoi n'est-il pas nécessaire d'écrire une condition d'arrêt dans un `foreach` ?

8.3. Transformer l'affichage

Créez `services.php` à partir de ce tableau :

```
<?php

$services = ["Apache", "PostgreSQL", "Redis", "Mailpit"];
```

Affichez chaque valeur sous la forme :

```
Service supervisé : Apache
```

CONTRAINTES:

- utilisez un seul `foreach` ;
- n'accédez pas aux éléments avec `$services[0]`, `$services[1]`, etc. ;
- placez une seule instruction `echo` dans la boucle.

8.4. Premier commit de la séance

```
git status
git add 04-boucles/tableaux.php 04-boucles/liste-serveurs.php 04-boucles/services.php
git commit -m "Ajoute le parcours des tableaux indexés"
git push
```

9. Activité 5 - Parcourir les clés et les valeurs

Pour un tableau associatif, `foreach` peut fournir simultanément la clé et la valeur.

Créez `charges-cpu.php` :

```
<?php

$chargesCpu = [
    "srv-web-01" => 45,
    "srv-bdd-01" => 82,
    "srv-cache-01" => 28
];

foreach ($chargesCpu as $nomServeur => $chargeCpu) {
    echo $nomServeur . " : " . $chargeCpu . " %<br>";
}
```

Dans l'expression :

```
foreach ($chargesCpu as $nomServeur => $chargeCpu)
```

- `$chargesCpu` est le tableau parcouru ;
- `$nomServeur` reçoit la clé courante ;
- `⇒` relie la clé à sa valeur ;
- `$chargeCpu` reçoit la valeur courante.



Dans la déclaration d'un tableau comme dans un `foreach`, `⇒` associe une clé à une valeur. L'opérateur `>=` signifie « supérieur ou égal ». Ne les confondez pas.

9.1. Construire une phrase

Créez `adresses-ip.php` avec :

```
<?php

$adressesIp = [
    "srv-web-01" => "192.168.10.11",
    "srv-bdd-01" => "192.168.10.12",
    "srv-cache-01" => "192.168.10.13"
];
```

Parcourez le tableau afin d'afficher :

```
Le serveur srv-web-01 utilise l'adresse 192.168.10.11.
```

QUESTION: Dans cet exercice, pourquoi une boucle sur les seules valeurs ferait-elle perdre une information utile ?

10. Activité 6 - Combiner `foreach` et conditions

Créez `alertes.php` :

```
<?php

$chargesCpu = [
    "srv-web-01" => 45,
    "srv-bdd-01" => 92,
    "srv-cache-01" => 28,
```

```
    "srv-fichiers-01" => 76
];

foreach ($chargesCpu as $nomServeur => $chargeCpu) {
    if ($chargeCpu >= 90) {
        echo $nomServeur . " : CRITIQUE<br>";
    } elseif ($chargeCpu >= 70) {
        echo $nomServeur . " : SURVEILLANCE<br>";
    } else {
        echo $nomServeur . " : NORMAL<br>";
    }
}
```

Avant d'exécuter le fichier, prévoyez l'état de chaque serveur.



Une version autonome de ce parcours avec comptage d'alertes se trouve dans `cours/sandbox-online/04-foreach.php` : copiez-collez-la sur https://3v4l.org pour l'exécuter sans Docker.

QUESTION: La condition est-elle évaluée une seule fois ou une fois pour chaque élément ?

11. Activité 7 - Compter, cumuler et calculer

Un compteur mémorise un nombre d'événements. Un accumulateur mémorise une somme.

Créez `statistiques.php` :

```
<?php

$chargesCpu = [
    "srv-web-01" => 45,
    "srv-bdd-01" => 92,
    "srv-cache-01" => 28,
    "srv-fichiers-01" => 76
];

$nombreServeurs = 0;
$nombreAlertes = 0;
$totalCharges = 0;

foreach ($chargesCpu as $nomServeur => $chargeCpu) {
    $nombreServeurs++;
    $totalCharges += $chargeCpu;

    if ($chargeCpu >= 70) {
        $nombreAlertes++;
        echo "Alerte sur " . $nomServeur . "<br>";
    }
```

```
}

$chargeMoyenne = $totalCharges / $nombreServeurs;

echo "Serveurs : " . $nombreServeurs . "<br>";
echo "Alertes : " . $nombreAlertes . "<br>";
echo "Charge moyenne : " . $chargeMoyenne . " %";
```

Complétez la trace.

Serveur traité	Nombre de serveurs	Nombre d'alertes	Total des charges
`srv-web-01`	…	…	…
`srv-bdd-01`	…	…	…
`srv-cache-01`	…	…	…
`srv-fichiers-01`	…	…	…

Répondez :

1. Pourquoi les trois variables sont-elles initialisées avant la boucle ?
2. Quelle variable est un accumulateur ?
3. Quelle variable permet de calculer la moyenne ?
4. Pourquoi la moyenne est-elle calculée après la boucle ?
5. `count($chargesCpu)` pourrait-il remplacer l'un des compteurs ?

12. Activité 8 - Diagnostiquer un parcours

Ouvrez le fichier fourni `diagnostic.php`. Il est syntaxiquement valide, mais ne produit pas le résultat demandé.

Le programme doit :

1. afficher chaque service, et non le tableau complet à chaque répétition ;
2. afficher le nom de chaque serveur avec sa charge ;
3. compter uniquement les charges supérieures ou égales à `70`.

Pour chaque erreur :

- prévoyez le résultat du code fourni ;
- identifiez le tableau, la clé et la valeur ;
- corrigez uniquement l'expression ou la variable fautive ;
- vérifiez le résultat dans le navigateur.

```
git diff -- 04-boucles/diagnostic.php
```

13. Exercice autonome - Produire un rapport de stockage

Créez `stockage.php` :

```
<?php

$espacesLibres = [
    "srv-web-01" => 42,
    "srv-bdd-01" => 8,
    "srv-cache-01" => 65,
    "srv-fichiers-01" => 12,
    "srv-sauvegarde-01" => 5
];
```

Pour chaque serveur, affichez son nom, son espace libre et son état :

- `CRITIQUE` si l'espace libre est inférieur à `10` Go ;
- `SURVEILLANCE` s'il est inférieur à `20` Go ;
- `NORMAL` dans les autres cas.

Affichez ensuite :

- le nombre total de serveurs ;
- le nombre de serveurs critiques ;
- l'espace libre total ;
- l'espace libre moyen.

CONTRAINTES:

- utilisez une seule boucle `foreach` ;
- utilisez la clé et la valeur ;
- utilisez une seule structure `if/elseif/else` ;
- utilisez au moins un compteur et un accumulateur ;
- calculez la moyenne après le parcours ;
- n'utilisez ni `for`, ni `while`, ni fonction créée par vos soins.

Préparez les résultats attendus sur papier avant l'exécution.

14. Travail sur papier

Sans ordinateur, étudiez :

```
<?php

$ports = [
    "HTTP" => 80,
    "HTTPS" => 443,
    "SSH" => 22
];

$total = 0;

foreach ($ports as $service => $numeroPort) {
    echo $service . " : " . $numeroPort . "<br>";
    $total += $numeroPort;
}

echo "Total : " . $total;
```

1. Donnez la valeur de `$service` à chaque répétition.
2. Donnez la valeur de `$numeroPort` à chaque répétition.
3. Donnez la valeur de `$total` après chaque répétition.
4. Écrivez exactement les quatre lignes affichées.
5. Que recevrait la variable de boucle avec `foreach ($ports as $numeroPort)` ?
6. Écrivez un `foreach` qui affiche uniquement les noms des services.

15. Exercices bonus

Facultatifs et gradués, dans le périmètre de la séance (`foreach`, tableaux, conditions, compteurs et accumulateurs ; ni `for`, ni `while`, ni fonction créée par vos soins). Autonomes : testez-les dans Docker ou sur https://3v4l.org. Le tableau de départ est :

```
<?php

$chargesCpu = [
    "srv-web-01" => 45,
    "srv-bdd-01" => 92,
    "srv-cache-01" => 28,
    "srv-fichiers-01" => 76
];
```

- `[*]` accessible ;

- [**] intermédiaire ;
- [*] défi.

15.1. Exercice bonus 1 [*] - Le serveur le plus chargé

En un seul `foreach`, trouvez le serveur dont la charge CPU est la plus élevée. Affichez :

```
Le plus chargé : srv-bdd-01 (92 %)
```

Indice : mémorisez au fil du parcours la plus grande charge rencontrée **et** le nom du serveur correspondant. Initialisez la charge maximale à une valeur très basse (par exemple `-1`).

15.2. Exercice bonus 2 [**] - Amplitude des charges

Toujours en un seul parcours, déterminez la charge la plus basse et la plus haute, puis l'**amplitude** (différence entre les deux). Affichez :

```
Min : srv-cache-01 (28 %)
Max : srv-bdd-01 (92 %)
Amplitude : 64 points
```

Indice : utilisez deux « mémoires » (min et max). Initialisez-les à `null` et remplacez-les lors du premier tour (`$min === null || $charge < $min`).

15.3. Exercice bonus 3 [*] - Défi : un rapport qui ne plante pas

La charge moyenne se calcule avec `$totalCharges / $nombreServeurs`. Mais que se passe-t-il si le parc est **vide** ?

1. Reprenez le calcul de moyenne de l'activité 7, mais placez d'abord `$chargesCpu = [];` (tableau vide) et exécutez. Notez le message d'erreur PHP obtenu.
2. Modifiez ensuite le programme pour qu'il affiche `Aucun serveur à superviser.` lorsque le parc est vide, et la charge moyenne sinon.
3. Vérifiez qu'il fonctionne dans les deux cas : tableau vide, puis tableau de charges rempli (moyenne attendue : `60.25 %`).

CONTRAINTE: pas de fonction créée par vos soins ; une seule structure `if/else` suffit à protéger la division.

16. Synthèse

Complétez.

1. Une variable qui stocke plusieurs valeurs est un …
2. Le premier indice d'un tableau indexé est …
3. Un tableau qui relie des clés à des valeurs est …
4. La fonction qui affiche la structure complète d'un tableau est …
5. La fonction qui compte les éléments d'un tableau est …
6. La boucle privilégiée pour parcourir un tableau est …
7. Dans `foreach ($serveurs as $serveur)`, `$serveur` représente …
8. Dans `foreach ($charges as $nom ⇒ $charge)`, `$nom` représente …
9. L'opérateur qui associe une clé à une valeur est …
10. Une variable qui mémorise une somme est un …

17. Validation Git

```
git status
git add 04-boucles
git commit -m "Ajoute les traitements avec foreach"
git push
git status
git log --oneline
```

Le dépôt doit être propre et les deux commits de la séance visibles sur la forge.

18. Arrêter le serveur PHP

En fin de séance, dans le terminal (vos fichiers restent) :

```
docker compose down
```